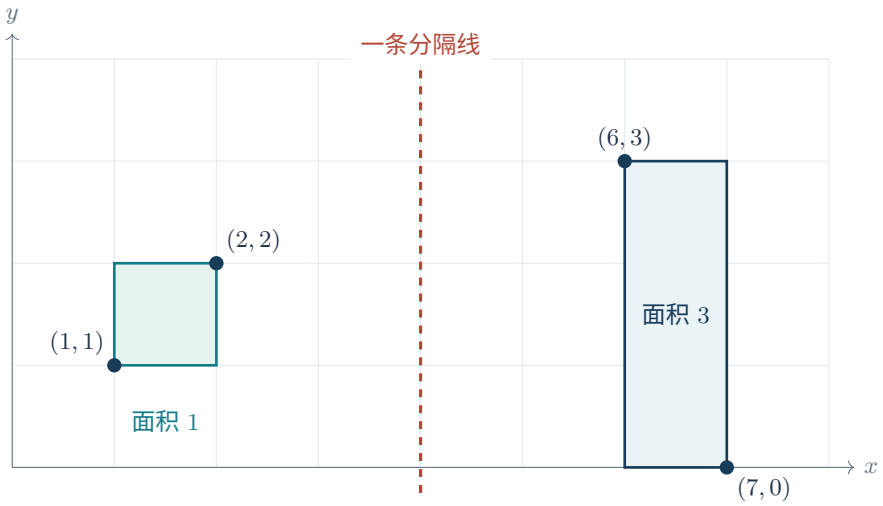
矩形覆盖

详细分析、证明与 C++14 代码

从“分隔线放在哪里”，到“递归如何得到最优答案”



核心问题：把所有点分给互不接触的轴对齐矩形，如何让面积和最小？
本册版本：$n \le 100$，$1 \le k < 4$；普通比较函数；`solve(vector<Point>& p, int k)`。

包含：几何图解・三矩形可分割证明・状态与递归・样例逐步推演
代码逐段分析・引用传参示意・复杂度・独立暴力核验与边界警告

阅读导航

先确认版本，再学习算法

这一点会影响算法是否正确

本册严格按照你贴出的 $k < 4$（**即最多 3 个矩形**）整理。洛谷 P1034 的公开题面是 $n \le 50,\ k \le 4$，与本次题面的范围不同。[1]
本册代码不能直接作为 $k = 4$ 版本的通用解法。第26页给出一个具体反例：真实答案为 0，直接套本算法会得到 24。

按你的提问安排的阅读路线

你想弄懂的内容	页码
题目到底要求什么；面积为什么不加 1	3–4
样例的两个矩形怎样画	5
两个、三个矩形为什么能用直线分开	6–8
为什么要排序；相同坐标为什么跳过	9–10
状态、转移、边界与正确性证明	11–13
样例六种分割与递归展开	14–15
这题与动态规划有什么关系	16
不用 Lambda；引用传参可以吗	17–18
完整代码与按行号分析	19–23
复杂度、测试、易错点与自测	24–28

符号约定

P 是一组具体的点，不只是点的数量；$S(P)$ 是用一个矩形覆盖它们的最小面积；$F(P,t)$ 是最多使用 t 个矩形的最小总面积。

图中的虚线表示“把平面分成两部分”的辅助线，**不是矩形的边，也不是必须输出的内容**。为展示位置关系，部分示意图不按比例绘制。

先读第5、8、18页建立直觉，再读证明和代码，会更容易把每个循环与图中的动作对应起来。

01 / 读懂题意

选的是覆盖方式，不是格子

给出平面上的 n 个点，每个点是一对整数坐标 (x_i, y_i)。你要用若干个矩形覆盖所有点，矩形数量不能超过给定预算 k，并让面积和最小。

限制	准确含义
边平行于坐标轴	矩形不能旋转；边只能水平或竖直。
覆盖所有点	每个点位于某个矩形内部或边界上。
矩形完全分开	不可重叠、不可碰边、不可碰角。退化矩形也遵守这一条。
允许面积为 0	一个点，或同一条水平线、竖直线上的点，可由零面积矩形覆盖。
输出内容	只输出最小面积和，不要求输出矩形坐标或分组方案。

本次题面给定的范围

$n \le 100$，$1 \le k < 4$，$0 \le x_i, y_i \le 500$。每个坐标的单位是几何长度，不是“格子个数”。

样例输入

```
4 2
1 1
2 2
6 3
7 0
```

样例输出

```
4
```

四个点，用两个矩形。
一组覆盖前两个点，另一组覆盖后两个点，面积分别是 1 和 3。

> 这不是前一题“矩阵中选互不相邻格子”。这里没有黑白染色、最大匹配，也没有“坐标差加 1”的计数规则。

两个常见误读

“完全分开”比“面积没有重叠”更严格：即使只共享一个角，也不允许。

“覆盖一条直线面积为 0”只指**水平线或竖直线**。斜线上两个点的最小轴对齐外接矩形通常有正面积。

02 / 第一个子问题

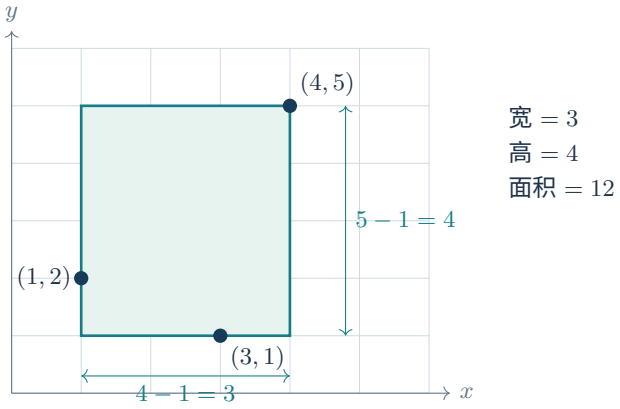
分好一组点，面积就确定了

只用一个矩形时，取四个极值即可

$$x_{\min} = \min x_i, \quad x_{\max} = \max x_i, \qquad y_{\min} = \min y_i, \quad y_{\max} = \max y_i.$$

最小外接矩形为 $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$，面积是

$$\boxed{S(P) = (x_{\max} - x_{\min})(y_{\max} - y_{\min})}$$



为什么一定最小?

任何覆盖这些点的矩形，横向必须包含最左和最右的点，宽至少是 $x_{\max} - x_{\min}$；纵向同理。四条边刚好贴住这些极值时，已经满足覆盖要求，因此这个下界能够达到。

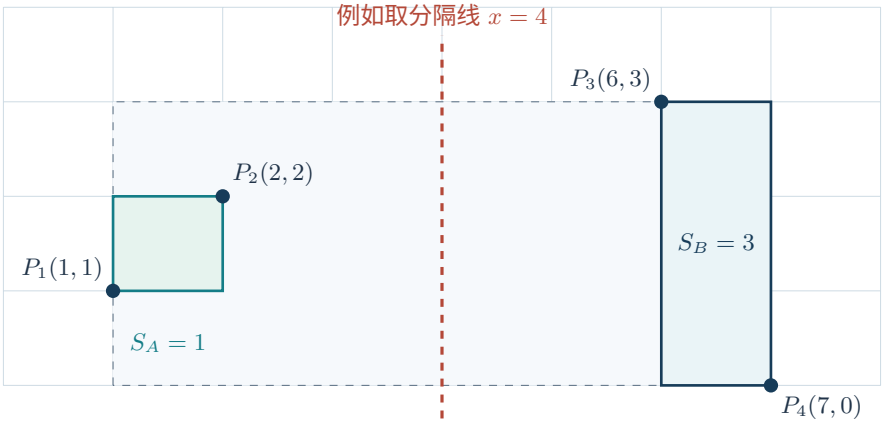
把某个覆盖矩形缩小为其负责点集的最小外接矩形，只会缩小范围。**原来互不接触的矩形，缩小后也不会产生新的接触。**

单个点	水平或竖直的一组点	两个点 $(1,1),(2,2)$
宽、高都为 0	至少一个方向跨度为 0	宽为 1，高为 1
面积 0	面积 0	面积 1，不是 4

03 / 样例图解

两个矩形，把 18 缩小到 4

如果四个点放在一个矩形中，横坐标范围为 $[1,7]$，纵坐标范围为 $[0,3]$，面积是 $6 \times 3 = 18$。



虚线在两组点之间的空隙里；矩形不需要贴着分隔线。

点组	横坐标范围	纵坐标范围	面积
$A = \{P_1, P_2\}$	$[1,2]$	$[1,2]$	1
$B = \{P_3, P_4\}$	$[6,7]$	$[0,3]$	3

这一个分割方案的总面积是 $S(A) + S(B) = 1 + 3 = 4$。
画出一个面积为 4 的方案，只证明答案不超过 4。要证明它最优，还需说明算法枚举了所有必要的分割，再比较面积；第14页会列出本样例全部候选分割。

算法要做的选择

决定“分隔线放在哪里”，就能决定哪些点属于左边、哪些属于右边。确定点组以后，各自的最小矩形由四个极值直接算出，不必逐条枚举矩形边界。

04 / 几何证明一

两个不接触的矩形，一定能切开

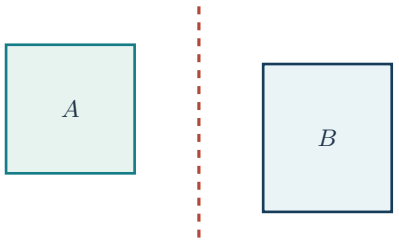
设两个闭矩形为

$$A = [a_1, a_2] \times [a_3, a_4], \qquad B = [b_1, b_2] \times [b_3, b_4].$$

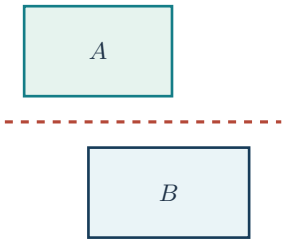
它们完全分开，当且仅当下面至少一个严格不等式成立：

$$a_2 < b_1 \quad \text{或} \quad b_2 < a_1 \quad \text{或} \quad a_4 < b_3 \quad \text{或} \quad b_4 < a_3.$$

前两种说明左右有空隙，后两种说明上下有空隙。



竖直分开：左右投影不相交



水平分开：上下投影不相交

反过来证明：如果两个方向都重叠会怎样？

如果横坐标区间有公共坐标 x_0，纵坐标区间也有公共坐标 y_0，那么 (x_0, y_0) 同时属于两个矩形。因此它们就有公共点，不满足完全分开。

所以，完全分开的两个矩形，至少有一个方向的投影严格分离；在这个空隙中放一条轴平行直线即可。

> 不等号必须是 $<$，不能改成 $\le$。例如 $a_2 = b_1$ 时，两矩形可能碰边。对零宽、零高的退化矩形，上述闭区间判断仍然成立。

这一步对算法的意义

只剩两个矩形时，不必枚举所有“每个点归谁”的组合。只需要尝试所有竖直分隔与水平分隔，每边一个最小外接矩形。

05 / 几何证明二

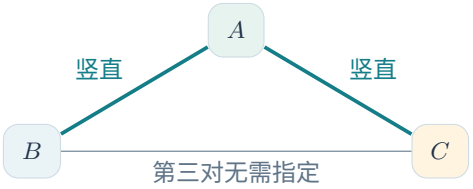
三个矩形：为什么总有第一刀？

命题

三个两两完全分开的轴对齐矩形，一定能用一条水平线或竖直线，把其中一个与另外两个严格隔开。

第一步：给每一对矩形选一个可分方向

三对关系是 AB, AC, BC。由上一页，每一对至少能竖直分开或水平分开。如果两个方向都可以，任选其一。三个关系只有两种标签，至少有两个关系标签相同。



这两个关系必然共用一个矩形。重命名后，可设为 AB 与 AC。先讨论它们都能竖直分开的情况。

第二步：沿横坐标，只会出现两类位置关系

同侧： B, C 都在 A 右边（都在左边同理）。那么

$$\max x(A) < \min\{\min x(B), \min x(C)\}.$$

在这个严格空隙中放一条竖直线，就能单独隔开 A。

两侧： 一个在左，一个在右。不妨是 B 在 A 左边、C 在右边，则横向顺序为 B, A, C。在 B 与 A 之间切开，可先隔开 B；也可在 A 与 C 之间先隔开 C。

第三步：另一个方向完全对称

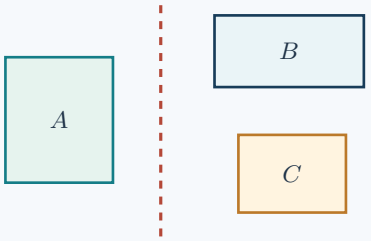
若共同标签是水平分开，把“左、右”换成“下、上”，证明相同。切完第一刀后，另外两个矩形继续由上一页的结论分开。命题得证。

> 这里的“同侧”指**整个矩形的坐标区间**都在另一侧，不是只比较中心点。这个证明不要求矩形等宽、等高，也允许退化矩形。

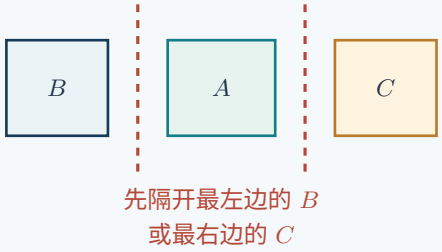
06 / 对应你提问的四幅图

同侧、两侧，分别怎样分？

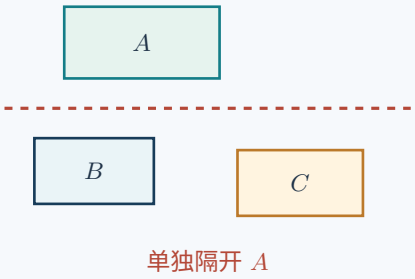
1. 竖直分开：B, C 在 A 同侧



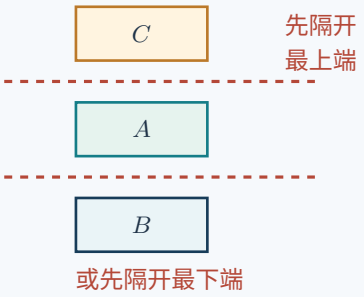
2. 竖直分开：B, C 在 A 两侧



3. 水平分开：B, C 在 A 同侧



4. 水平分开：B, C 在 A 两侧



图中画出的两条候选虚线是“二选一”，不是一次必须切两刀。

> 先隔开谁并不固定；代码通过枚举方向与切分位置，自动覆盖这些情况。第一刀竖直、第二刀水平也完全允许。

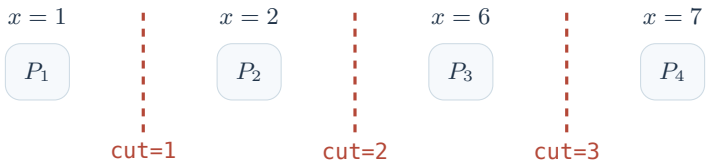
07 / 从几何到循环

不用枚举实数坐标，只枚举点间空隙

竖直分割：先按横坐标排序

样例排序后是

$$P_1(1,1),\ P_2(2,2),\ P_3(6,3),\ P_4(7,0).$$



在同一个空隙里移动分隔线，不会改变两边分别有哪些点，面积计算结果也不变。例如 $x=3$、4、5 都得到同一个分组。因此每个严格空隙只需考察一次。

```
sort(p.begin(), p.end(), cmpX);
for (int cut = 1; cut < s; cut++) {
    // A 是前 cut 个点，B 是剩余的点
}
```

水平分割：改为按纵坐标排序

此时样例顺序为 P_4, P_1, P_2, P_3，同样枚举相邻不同纵坐标之间的空隙。代码中的 `dir` 负责选择排序方向。

方向	排序	形成的两组
`dir=0`	`cmpX`：横坐标递增	左边与右边
`dir=1`	`cmpY`：纵坐标递增	下边与上边

> `cut=2` 表示“前 2 个点划入第一组”，**不是**分隔线坐标等于 2。线可以放在第二、第三个点之间的任何严格空隙里。

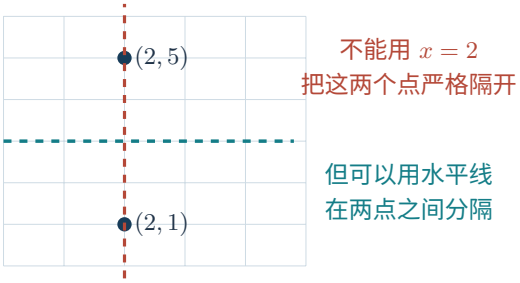
为什么每个方向最多 $s-1$ 个候选？

s 个排序后的点只有 $s-1$ 个内部相邻间隙。首尾之外的线会使一边为空，不需要枚举；相同坐标处的间隙还要跳过，实际候选可能更少。

08 / 不能省略的判断

同一横坐标，为什么不能竖着切?

例如两点为 $(2,1)$ 和 $(2,5)$。它们横坐标都等于 2，没有任何竖直线能使它们分别位于严格的左右两侧。




```
if (dir == 0 && p[cut - 1].x == p[cut].x)
    continue;
if (dir == 1 && p[cut - 1].y == p[cut].y)
    continue;
```

这两个判断到底保证了什么?

若横坐标已排序且在不同坐标之间分组，就有

$$\max_{v\in A} v.x < \min_{v\in B} v.x.$$

第一组的所有最小外接矩形都在左侧，第二组都在右侧，中间始终保留严格空隙。以后各组内部继续分割，也不会跨过这个空隙。

> **先保证构造合法，再计算面积。**因此主算法不需要在每次得到子答案后，再逐对检查矩形是否相交。

“跳过这个方向”不等于“这两点永远不能分开”

横坐标相同的点，仍可能被水平线分开。算法会枚举两个方向，所以不会因为跳过某个竖直切分而漏掉所有可能的分法。

若两个点坐标完全相同，它们不可能被两个互不接触的非空矩形分别覆盖；应让它们由同一个矩形负责。

09 / 状态与转移

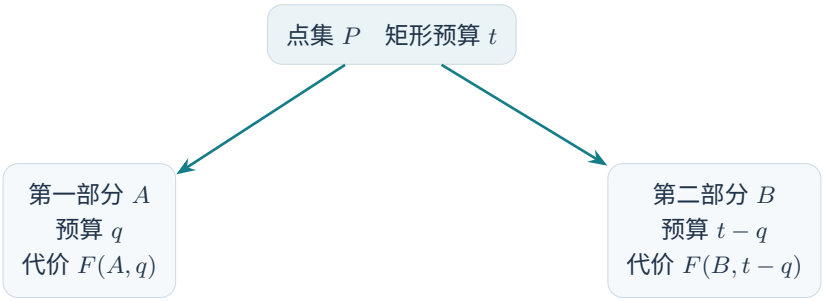
把几何问题写成一个子问题

> $$\boxed{F(P,t) = \text{至多用} t \text{ 个互不接触的矩形覆盖点集} P \text{ 的最小面积和}}$$
>
> `solve(p, k)` 就是在计算这个值。P 必须说明“是哪几个点”，而不是只说明“有几个点”。

一个状态有两类候选方案

不继续分：直接用一个最小外接矩形，代价为 $S(P)$。

继续分：选择一个合法空隙，将点集分为两个非空集合 A, B，再给第一部分分配 q 个矩形、第二部分分配 $t-q$ 个矩形。



$$F(P,t) = \min\left\{ S(P),\ \min_{\substack{\text{合法轴向切分}(A,B)\\ 1\le q<t}} \left[F(A,q) + F(B,t-q)\right] \right\}.$$

为什么里面是加法，外面是最小值?

两个点组**都要**覆盖，所以这一个方案的面积要相加；不同分割方案之间只需选**最好的一种**，所以取最小值。

```
for (int t = 1; t < k; t++) {
    ll leftArea = solve(A, t);
    ll rightArea = solve(B, k - t);
    ans = min(ans, leftArea + rightArea);
}
```

这里代码里的 `t` 是给第一组的预算，对应公式中的 q；参数 `k` 对应公式里的总预算 t。两者不要混淆。

10 / 边界与语义

为什么有些情况直接返回?

代码判断	对应的最简单子问题
`if (s <= k) return 0;`	矩形预算足够，可以分别覆盖每个不同位置的点；重复点合在同一个零面积矩形。
`if (k == 1) return ans;`	只有一个矩形，没有分割自由度，答案就是外接矩形面积。
`if (ans == 0) return ans;`	面积不能为负，已经得到 0，不可能继续改进。

“至多 k 个”与题目的“k 个”

本册把 k 解释为可使用的矩形预算。如果题目要求**恰好 k 个非空矩形**，并且至少有 k 个不同位置的点，最优面积与“至多 k 个”相同。

为什么可以补足数量?

若某个方案用了少于 k 个非空矩形，且不同点的总数至少为 k，其中至少有一个矩形负责两个不同的点。这些点至少在一个坐标方向上不同，可在相邻不同坐标之间分成两组。
两个新外接矩形都位于原矩形范围内，彼此有严格空隙，也不接触其他矩形。若竖直分割，新矩形的宽度和不超过原宽，高度各不超过原高，因此总面积不增加。水平分割同理。
重复此操作，直到得到恰好 k 个非空矩形。

> 若题目同时允许重复坐标，却要求恰好 k 个非空矩形，并且不同位置少于 k，问题语义就需要额外说明。不要用“预算足够就返回 0”去声称这种精确数量约束也一定可行。

递归一定能停下来吗?

每次切分的两边都非空；矩形预算分成 $1 \le q < t$ 和 $t - q$，两边都严格少于原预算。预算最终降到 1；或者点很少、面积已经为 0，提前结束。不存在无限递归。

11 / 完整正确性

为什么算法既不乱算，也不漏算?

证明针对 $1 \le k \le 3$，并按矩形预算归纳。

基础：预算只有 1 个

算法返回点集的最小外接矩形面积，已经在第4页证明最优。点数不多于预算或外接面积为 0 的提前返回，也由非负性及具体零面积方案保证正确。

第一方向：算法返回的方案一定合法

每一次切分都在严格的坐标间隙中。根据归纳假设，两边分别得到合法的覆盖；所有左侧矩形都只需位于左侧点集的范围内，右侧同理。

两组之间存在严格空隙，因此合并两边的方案后，仍然互不接触，矩形总数也不超过预算。直接使用一个矩形的初始方案同样合法。

$$\text{算法最小值} \ \ge \ \text{真实最优值}.$$

第二方向：任意最优方案都能被枚举到

取一个最优方案，先把每个矩形缩到其负责点集的最小外接矩形。这不会增加面积，也不会产生重叠。

若只有一个非空矩形，初始候选 $S(P)$ 已经涵盖它。若有两个或三个矩形，由前面的几何证明，一定存在第一条水平或竖直分隔线，将它们分成两个非空部分。

排序后，这条线对应某个相邻不同坐标之间的空隙，算法会枚举到。两侧的矩形数量都为正，且总数不超过预算，所以也能被某个 `t` 与 `k-t` 的分配涵盖。归纳假设保证两边递归最优值不会比该方案的对应部分更大。

$$\text{算法最小值} \ \le \ \text{真实最优值}.$$

> 两个方向合起来得到： **算法返回值 = 真实最优值。**
> 证明中最关键的桥梁，不是“用了递归”，而是**最多三个矩形一定存在第一条可分隔直线**。

关于子问题独立性

两侧的解只需各自在其点集范围内收缩，因此不会因某一侧选择了更好的内部划分，而影响另一侧的合法性。这正是可以分别求最优再相加的原因。

12 / 手算样例

六个候选分割，逐一比较

使用两个矩形时，每条分隔线的两边各用一个矩形。样例中所有横、纵坐标都不同，因此每个方向各有三个候选。

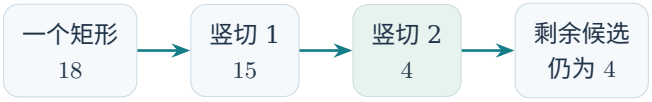
按 x 排序：P_1, P_2, P_3, P_4

cut	分组	$S(A)$	$S(B)$	合计
1	$\{P_1\} \mid \{P_2, P_3, P_4\}$	0	15	15
2	$\{P_1, P_2\} \mid \{P_3, P_4\}$	1	3	**4**
3	$\{P_1, P_2, P_3\} \mid \{P_4\}$	10	0	10

按 y 排序：P_4, P_1, P_2, P_3

cut	分组	$S(A)$	$S(B)$	合计
1	$\{P_4\} \mid \{P_1, P_2, P_3\}$	0	10	10
2	$\{P_4, P_1\} \mid \{P_2, P_3\}$	6	4	10
3	$\{P_4, P_1, P_2\} \mid \{P_3\}$	12	0	12

对应 ans 的变化



一项展开：第一种竖切为什么是 15？

右边三个点是 $(2,2), (6,3), (7,0)$，横向跨度 $7-2=5$，纵向跨度 $3-0=3$，面积为 15；左边只有一个点，面积为 0。所以总面积是 15。

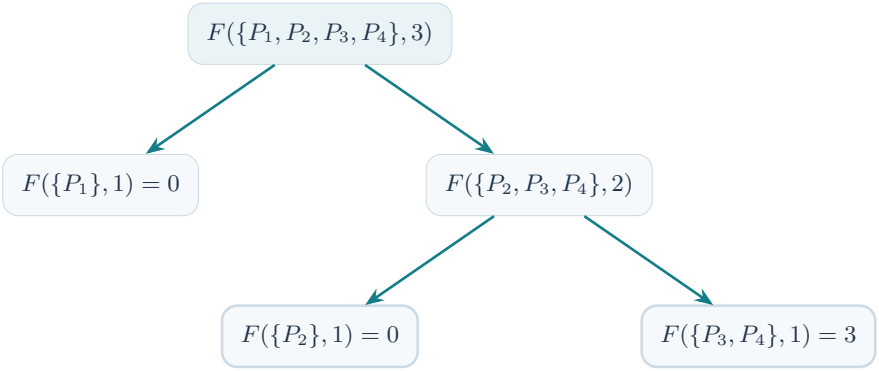
最优值 4 不是因为程序“恰好猜到第二刀”，而是比较了所有必须考虑的方向与空隙；每个分组的面积由极值唯一确定。

13 / 递归展开

把同样四个点的预算改成 3

这一页是**扩展示例**，把样例中的 $k = 2$ 改成 $k = 3$。它用于看清递归层次，不是原样例的输出。

第一刀若把 P_1 单独分到左边，给左边 1 个矩形、右边 2 个，则会进入一个真正需要继续分割的子问题。



图中只画其中一个候选分割分支，程序还会检查其他方向与位置。

这一支得到 $0 + (0 + 3) = 3$。但另一个首刀会更好：

$$\{P_1, P_2\} \mid \{P_3, P_4\}, \qquad F(\{P_1, P_2\}, 1) + F(\{P_3, P_4\}, 2) = 1 + 0 = 1.$$

为什么扩展示例的最优答案是 1？

四个点的横坐标彼此不同，纵坐标也彼此不同。只用三个非空矩形，至少一个矩形要覆盖两个点；任意两个点的宽、高都是正整数，面积至少为 1。上面的方案达到 1，因此它最优。

> 递归中的每一次返回值，都是**当前点集、当前预算**的最优解。外层把两边相加，再和其他候选比较；不能把树上所有分支的结果全部加起来。

两个循环枚举的是不同东西

`cut` 决定“哪些点属于哪一边”；`t` 决定“每一边最多用几个矩形”。$k = 3$ 时，必须考虑 $1 + 2$ 与 $2 + 1$，因为两侧点集通常不同，不能只保留其中一个。

14 / 回答你的追问

这题是不是动态规划？

可以用状态转移分析，但这份代码没有记忆化

我们已经写出 $F(P,t)$ 及其转移，所以它具备动态规划式的子问题描述。**递归**只是组织求解过程的一种方式；它本身不意味着有没有动态规划。

层面	本册中的含义
问题拆分	用水平或竖直线把点集分为两个子问题。
最优值组合	两部分最优面积相加，不同划分取最小。
本册实现	递归分治枚举，遇到同一状态可能重新计算。
加入状态缓存	同一 $F(P,t)$ 只计算一次，其余直接复用，可写成记忆化搜索。

为什么不能只写 `dp[点数][矩形数]`？

同样两个点、一个矩形：

$$\{(0,0),(1,1)\} \Rightarrow 1, \qquad \{(0,0),(10,10)\} \Rightarrow 100.$$

两个状态的点数和预算相同，但位置不同，答案不同。状态必须包含“是哪一组点”的信息。

若要记忆化，怎样辨认同一个状态？

可给每个输入点一个固定编号，将“排序后的点编号集合”和矩形预算一起作为键。每个状态的标识必须与递归中的坐标排序无关：同一组点换了顺序，仍应命中同一份缓存。

> $n = 100$ 时不能把所有点集简单压进一个 32 位或 64 位整数，也不能枚举所有 2^n 个子集。记忆化只有在重复状态足够多、键的处理成本合适时才有实际帮助。

为什么本册保留递归版本？

在 $k \le 3$ 的限定下，当前实现最坏只需要 $O(n^3)$ 时间，结构清楚、便于证明，也直接对应你正在学习的函数和循环。本册不把“加缓存”包装成必要步骤或保证更快的优化。

15 / 语法拆解一

不用 Lambda，改成普通比较函数

```
bool cmpX(Point A, Point B) {
    if (A.x != B.x) return A.x < B.x;
    return A.y < B.y;
}

bool cmpY(Point A, Point B) {
    if (A.y != B.y) return A.y < B.y;
    return A.x < B.x;
}
```

`cmpX(A,B)` 返回 `true`，表示 A 应排在 B 前面：先比较横坐标；横坐标相等时，再比较纵坐标。`cmpY` 交换两种坐标的优先级。

```
if (dir == 0) {
    sort(p.begin(), p.end(), cmpX);
} else {
    sort(p.begin(), p.end(), cmpY);
}
```

为什么写 `cmpX`，不是 `cmpX()`？

这里是把比较函数交给 `sort`；排序过程中由 `sort` 反复传入两个点进行比较。它处理的是左闭右开的范围 `[p.begin(), p.end())`。[3]

坐标一样时，返回什么？

如果两个点的横、纵坐标都一样，比较结果是 `false`。不能把小于号改成小于等于号；比较函数必须表达“严格排在前面”，不能说一个元素严格排在自己前面。

> `Point A, Point B` 按值复制两个点。一个点只有两个整数，这样写最直观。也可以改成 `const Point&` 只读引用，但与主递归传整个 `vector` 的开销不是同一个量级。

这次修改改变了什么？

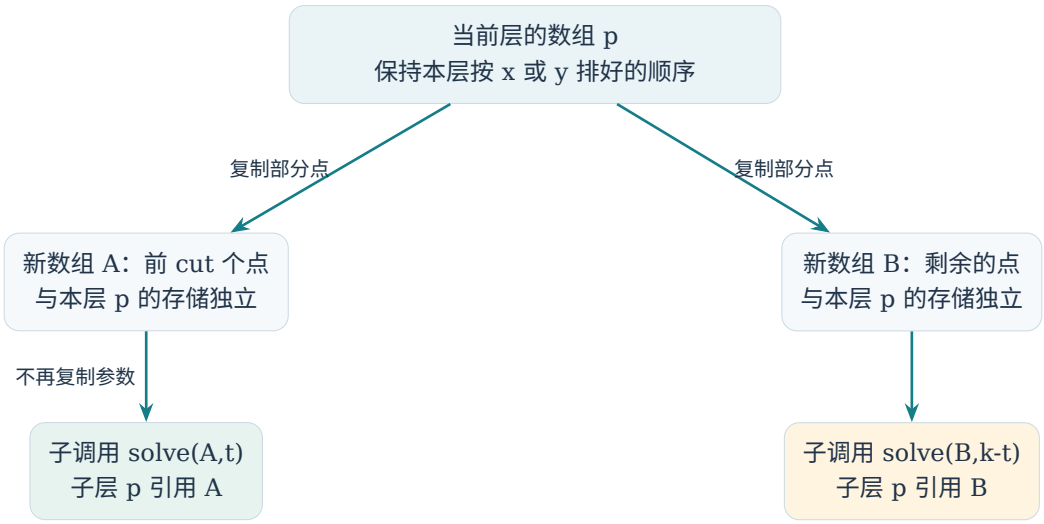
只是把排序规则从 Lambda 移到了两个普通函数里。方向、候选分割、递归转移与正确性证明都没有改变。

16 / 语法拆解二

引用传参，为什么不会打乱外层?

```
ll solve(vector<Point>& p, int k)
```

引用是原对象的另一个名字。调用 `solve(A,t)` 时，参数 `p` 直接代表 A；在函数里排序 `p`，就会改变 A 的顺序，不会再复制一份参数数组。[2]




```
vector<Point> A(p.begin(), p.begin() + cut);
vector<Point> B(p.begin() + cut, p.end());
```

这两行调用范围构造函数，为 A、B 建立独立的数组并复制对应点。[4] 所以子递归重新排序 A、B，不会改变**当前这一层**的 p。

同一个 A 在不同预算下被反复调用，会受影响吗?

不会影响答案。排序只改变排列顺序，没有改变点集；需要继续分割时函数会重新排序。**但若你把子问题改成直接引用父数组的一段共享存储，就不能照搬这个安全结论。**

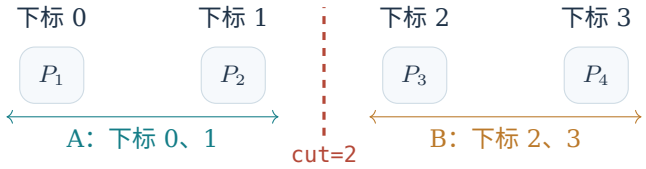
> `area(const vector<Point>& p)` 只读，不复制数组。
> `solve(vector<Point>& p,...)` 允许排序，不能加 `const`。
> 调用仍写 `solve(A,t)`，不是 `solve(&A,t)`。

17 / 先认变量再读源码

三个循环，每层只负责一件事

名称	含义
`p`	本次递归要覆盖的具体点集。
`k`	本次递归最多可使用的矩形数量。
`s`	当前点集大小，不一定等于最初输入的 n。
`ans`	当前状态已经找到的最小面积和。
`dir`	0：按 x 排序后竖切；1：按 y 排序后横切。
`cut`	第一组有多少个点，取值 1 到 s-1。
`t`	给第一组多少个矩形，取值 1 到 k-1。
`A, B`	这一次切分产生的两个独立子数组。

范围构造函数是左闭右开的




```
vector<Point> A(p.begin(), p.begin() + cut);
// 包含 begin，不包含 begin + cut
vector<Point> B(p.begin() + cut, p.end());
// 包含 begin + cut，不包含 end
```

对应“方向 → 位置 → 预算”

先选横切还是竖切；再选在哪个间隙切；最后决定两侧分别用几个矩形。每个合法组合形成一个候选答案，`min` 只保留最小的那个。

> 同样叫 A、B，在几何证明里可能代表矩形，在代码中代表点集。代码是**先分点集，再由点集确定最小矩形**。

18 / 完整 C++14 源码 · 1/3

数据结构、比较函数、面积计算

以下行号与配套 `rectangle_cover_illustrated.cpp` 完全一致。代码保留你最后选择的普通比较函数和引用传参，不使用 Lambda。

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  using ll = long long;
7
8  struct Point {
9      int x, y;
10 };
11
12 // 按横坐标排序；横坐标相同，再按纵坐标排序。
13 bool cmpX(Point A, Point B) {
14     if (A.x != B.x) return A.x < B.x;
15     return A.y < B.y;
16 }
17
18 // 按纵坐标排序；纵坐标相同，再按横坐标排序。
19 bool cmpY(Point A, Point B) {
20     if (A.y != B.y) return A.y < B.y;
21     return A.x < B.x;
22 }
23
24 // 用一个矩形覆盖点集 p 的最小面积。
25 ll area(const vector<Point>& p) {
26     if (p.empty()) return 0;
27
28     int minX = p[0].x, maxX = p[0].x;
29     int minY = p[0].y, maxY = p[0].y;
30
31     for (const Point& v : p) {
32         minX = min(minX, v.x);
33         maxX = max(maxX, v.x);
34         minY = min(minY, v.y);
35         maxY = max(maxY, v.y);
36     }
37
38     return 1LL * (maxX - minX) * (maxY - minY);
39 }
```

先抓住这三个入口

第 8 行保存一个点；第 13、19 行规定排序；第 25 行只负责“一组已确定的点，用一个矩形面积是多少”。

19 / 完整 C++14 源码 · 2/3

核心递归：方向、切点、预算

```cpp
// 至多用 k 个互不接触的矩形覆盖点集 p。
// 按题面 k < 4，即 k <= 3 的条件使用。
ll solve(vector<Point>& p, int k) {
    int s = static_cast<int>(p.size());

    if (s <= k) return 0;

    ll ans = area(p);
    if (k == 1 || ans == 0) return ans;

    for (int dir = 0; dir < 2; dir++) {
        // dir=0：竖直分割；dir=1：水平分割。
        if (dir == 0) {
            sort(p.begin(), p.end(), cmpX);
        } else {
            sort(p.begin(), p.end(), cmpY);
        }

        // 前 cut 个点分为一组，其余点分为另一组。
        for (int cut = 1; cut < s; cut++) {
            // 同一坐标处不能用对应方向的直线严格分开。
            if (dir == 0 && p[cut - 1].x == p[cut].x) continue;
            if (dir == 1 && p[cut - 1].y == p[cut].y) continue;

            // A、B 都是独立的新数组，不是 p 的引用。
            vector<Point> A(p.begin(), p.begin() + cut);
            vector<Point> B(p.begin() + cut, p.end());

            for (int t = 1; t < k; t++) {
                ll leftArea = solve(A, t);
                ll rightArea = solve(B, k - t);
                ans = min(ans, leftArea + rightArea);
            }
        }
    }

    return ans;
}
```

核心是第 69–72 行

两侧必须同时覆盖，所以面积相加；不同选择之间取最小。递归对子数组排序不影响父层 p，原因见第18页。

20 / 完整 C++14 源码 · 3/3

读入点集，输出根状态

```cpp
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;

    vector<Point> p(n);
    for (int i = 0; i < n; i++) {
        cin >> p[i].x >> p[i].y;
    }

    cout << solve(p, k) << '\n';
    return 0;
}
```

为什么输出 `solve(p,k)`？

主函数中的 p 包含全部输入点；k 是全部矩形预算。因此这个调用对应整个题目，而不是某个局部子问题。

源码导航

行号	阅读重点
1–10	标准头文件、`long long` 别名、点结构体。
13–22	不用 Lambda 的两个比较函数。
25–39	扫描极值，乘积先提升到 `long long`。
43–49	当前状态、三种提前返回。
51–63	两个方向、全部切点、严格分隔条件。
66–72	创建独立子数组，递归两侧，更新最优值。
80–94	输入、最终答案与标准输出。

编译与运行

```bash
g++ -std=c++14 -O2 -Wall -Wextra \
    rectangle_cover_illustrated.cpp -o rectangle_cover
./rectangle_cover < input.txt
```

本题提供的输入输出格式没有指定额外文件名，配套程序使用标准输入输出。若实际评测另行要求文件名，应按该平台说明修改，而不是随意添加 `freopen`。

> **支持范围：** $1 \le k \le 3$。程序不会自动识别同名题是否是另一份约束；提交前必须核对题面。

21 / 把源码读成中文

从进入 solve 到 return ans

第 43-49 行：先处理不需要搜索的情形

s 是当前点数。矩形预算足够，返回 0；否则先用一个矩形得到合法上界 ans=area(p)。只剩一个矩形，或上界已经为 0，就无需继续枚举。

ans 不是初始化为 0。它保存“当前找到的最小值”，若一开始就设成 0，再与非负面积取最小，将永远得到 0。

第 51-63 行：先让划分有几何意义

排序把“左半边或下半边”变成数组的前缀，cut 才能表示一条分隔线。若相邻坐标一样，跳过对应方向的切分，不让两个组只靠人为的数组顺序分开。

第 66-67 行：把子问题的数据隔离

A、B 是新数组。它们会反复传给 solve，顺序可能被改变，但点的成员不变。父层 p 继续保有本轮 dir 所需的排序。

第 69-72 行：独立求最优，再组合

```
ll leftArea = solve(A, t);
ll rightArea = solve(B, k - t);
ans = min(ans, leftArea + rightArea);
```

第一行先算完 A 的最优值再返回；第二行处理 B；第三行只比较这一种“切分位置 + 预算分配”与过去最优值。leftArea/rightArea 在横切时实际对应下、上两部分，变量名不改变含义。

第 77 行：所有必要候选都比较完后返回

返回的是本状态的一个整数最优值，不是“所有分支的和”，也不要求返回点的原排列。上一层继续把这个数加入它自己的候选方案中。

> 不能在发现第一个合法切分后就 return；那只得到一个可行方案。只有返回 0 等“已经证明不能更小”的情形，才可以安全提前结束。

为什么代码不记录矩形坐标?

题目只要求最小面积。若还需要输出具体方案，应额外记录每个状态最优的切分、方向和预算分配，再恢复叶子点组；不影响本题的数值计算思路。

22 / 复杂度与优化

三层限制带来 $O(n^3)$ 上界

把当前有 s 个点、预算为 t 的耗时记作 $T_t(s)$。

预算	主要工作	本实现上界
1	扫描极值，计算外接矩形	$T_1(s) = O(s)$
2	$O(s)$ 个空隙，每次复制并扫描两组	$T_2(s) = O(s^2)$
3	每个空隙有 $1+2$ 或 $2+1$，其中一侧需处理两矩形子问题	$T_3(s) = O(s^3)$

把第三行展开一次

忽略常数方向和两种预算分配，主要计算量满足

$$\sum_{c=1}^{s-1} \big(O(c^2) + O((s-c)^2) + O(s)\big) = O(s^3).$$

每层还有 $O(s \log s)$ 的排序；在上述上界中不占主导。`sort` 的比较次数上界为 $O(s \log s)$。[3]

空间为什么不是指数?

候选分割是依次计算，不是把所有递归树同时存下来。每次只保留当前递归路径上的 p、A、B；在 $k \le 3$ 下，递归深度是常数，每层保存的点总数为 $O(n)$，额外空间为 $O(n)$。

> 引用传参减少了“调用时再复制参数”的常数开销。**它没有消除创建 A、B 的复制，也没有把 $O(n^3)$ 改成更低的复杂度。**

进一步优化可以从哪里入手?

两矩形子问题里，排序后可预处理每个前缀、后缀的四个极值。这样每个切分的两边面积可 $O(1)$ 查询，使两矩形子问题降为 $O(s \log s)$（排序主导）。把这个专门优化代入三矩形求解，可得到 $O(n^2 \log n)$ 级别的实现。

这是进一步改写的方向，**本册配套代码没有实现该优化**。对 $n \le 100, k \le 3$，先掌握当前版本及其证明更重要。

23 / 本地核验记录

证明负责一般情况，测试负责查实现

本次整理重新编译了配套源码，并编写独立的暴力程序核对，不沿用历史回答中未经本次复核的测试数字。

测试类别	数量
3×3 坐标网格全部非空点子集；每组 $k = 1, 2, 3$	1,533
随机小规模点集，$n \le 9$，坐标在 $[0, 10]$	3,000
同横坐标、同纵坐标、重复坐标等退化数据	600
本题原样例	1
与独立暴力答案逐一比较	**5,134**
$n = 100$ 的随机规模与单调性检查	60

独立暴力到底枚举什么？

按固定点顺序，枚举把每个点加入已有组或新建一组（总组数不超过 k）。每组维护最小外接矩形，若两个闭矩形不满足严格分离条件，就判为冲突。最后比较所有合法分组的面积和。

它不使用“必须先有一条分隔线”的假设，所以能够独立核对本册切分算法。部分框相交后再增添点，只会扩大框，不可能消除相交，因此这种冲突剪枝是安全的。

本次结果

5,134 组小规模比较全部一致。原样例输出 4。**AddressSanitizer 和 UndefinedBehaviorSanitizer** 下重新运行同一批测试及规模检查，未报告异常。

这只是本地验证记录，**不是线上 AC 记录，也不能替代前面的数学证明**。

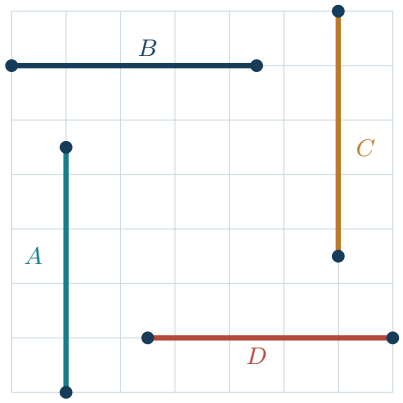
提交前手工检查

一个点；全部点水平或竖直共线；$k = 1$；矩形预算足够；重复横坐标；两矩形恰好可能碰边；$k = 3$ 时需要先横后竖的情况。
对同一组点，预算增加时，最小面积不应增加；答案应非负，且不超过全部点一个外接矩形的面积。

24 / 必须保留的适用范围

为什么不能直接把 k 改成 4?

洛谷 P1034 的公开约束允许 $k \le 4$。[1] 但“三个矩形总能先切开一个”的证明，不能直接推广到四个矩形。下面是本文构造并独立验证的反例。



$A : (2, 0), (2, 9)$
$B : (0, 12), (9, 12)$
$C : (12, 5), (12, 14)$
$D : (5, 2), (14, 2)$

4 个互不接触的线段矩形
总面积 $= 0$

没有第一条贯穿的分隔线

这些矩形的横向投影合起来覆盖整个区间 $[0, 14]$，纵向投影也覆盖 $[0, 14]$。任何能把它们分成两侧的水平或竖直线，都会碰到至少一个矩形，无法沿空隙先分开。

并且这 8 个点中，只有图中的四对共享相同的横坐标或纵坐标，没有三点共水平或竖直线。要用至多 4 个零面积矩形覆盖 8 个点，必须正好使用这四对；不存在另一套可逐次直切的零面积方案。

> 真实答案是 0。把配套程序的输入改成这 8 个点且 $k = 4$，程序会返回 24。这不是浮点精度问题，而是**枚举范围没有涵盖非直切的布局**。本次独立暴力也得到真实答案 0。

如果你的实际评测允许 $k = 4$，需要重新选择能覆盖一般布局的搜索、状态设计或补充分支；不能仅修改循环上限。本册仍以你提供的 $k < 4$ 题面为准。

25 / 易错点与自测

把“会写”变成“能检查”

容易写错的地方	应怎样检查
面积写成坐标差加 1	几何长度不加 1；单点面积必须为 0。
`ans` 初始化为 0	初始候选取 `area(p)`，再不断取更小值。
只尝试竖直切分	水平切分也必须枚举，子递归方向也可改变。
只考虑 $1+2$	对 $k=3$，还要考虑 $2+1$。
相同坐标处也按数组切	数组相邻不等于几何上存在严格空隙。
认为引用后无任何复制	A、B 的范围构造仍复制点；只省参数复制。
`area` 能为负	极值初始化为第一个点，并正确取 min/max。
套到 $k=4$ 版本	第26页反例说明会漏掉布局。

自测 1：四个角

点为 $(0,0),(0,2),(2,0),(2,2)$。只用一个矩形面积是多少？用两个呢？

答案：一个面积 4；两个可以分别覆盖左右两列，各为零面积竖线，面积和 0。

自测 2：对角线上三个点

点为 $(0,0),(1,1),(2,2)$，$k=2$。

答案：1。一个矩形覆盖相邻两点，面积 1；另一个覆盖剩余单点，面积 0。不能因为三点共斜线，就说一个轴对齐矩形面积为 0。

自测 3：引用会改谁？

本层先创建 A、B，再调用 `solve(A,2)`。子函数排序参数 `p`，会不会把本层 `p` 也重排？

答案：不会。子层 `p` 引用 A，A 是独立的新数组。会变化的是 A 的顺序，不是父层 `p` 的顺序。

> 解释一个循环时，至少说出三件事：它枚举什么、边界为何这样写、每次迭代产生什么候选答案。只会背函数名还不够。

26 / 复习卡片与参考

把这道题压缩成一条思考链

确定点组 ⟶ 四个极值得到最小外接矩形。
两个矩形 ⟶ 至少有一个坐标投影严格分离。
三个矩形 ⟶ 三对关系中两对方向相同，必有第一条分隔线。
枚举实现 ⟶ 排序方向、不同坐标间的切点、两边矩形预算。
递归合并 ⟶ 两边最优面积相加，所有方案取最小。
代码细节 ⟶ 普通比较函数；子数组独立；引用参数省去额外复制。

复习时应能独立回答

为什么面积不加 1？为什么相同横坐标不能竖切？三个矩形的证明中“共用一个矩形”从哪里来？为什么只有点数不能作为 DP 状态？为什么 `solve` 可以使用引用而 `area` 适合只读引用？为什么不能把证明直接搬到 $k = 4$？

资料来源与使用范围

[1] 洛谷 P1034：NOIP 2002 提高组《矩形覆盖》。用于核对同名题的规则及版本差异；该页面约束为 $n \le 50, k \le 4$，不是本册代码的适用范围。

[2] C++ 工作草案：References [dcl.ref]。用于引用类型及引用参数修改原对象的语义说明。正文示例采用 C++14 已支持的语法。

[3] C++ 工作草案：Sorting [alg.sort]。用于 `sort` 的比较函数、排序范围与比较次数说明；不使用文档中新标准专有接口。

[4] C++ 工作草案：vector constructors [vector.cons]。用于范围构造函数及元素复制行为说明。

本次整理依据

题面、坐标样例、你对动态规划／图示／普通比较函数／引用传参的追问，以及已提供的源文件：

`rectangle_cover_reference.cpp`。

本册配套源码与其算法一致；几何证明、反例、推演图和独立测试由本册重新整理。

核验日期：2026-10-23。所有示意图为本册绘制；源码可单独下载。

学习重点不是记住三层循环，而是证明：为什么这些循环已经覆盖全部必要方案。